

9.5 - Versionamento de APIs

Evoluir o contrato sem quebrar quem depende

O custo de uma breaking change

Seu contrato, o sistema dos outros

- **API pública é contrato com consumidores que você não controla**
 - Cada integração depende do formato exato da resposta
- **Mudar o contrato sem aviso quebra sistemas em produção**
 - O dano não é seu, é de quem confiou na sua API
- **Breaking change exige planejamento, não só implementação**
- **Compatibilidade de longo prazo é disciplina de governança**
 - A pergunta não é 'posso mudar', é 'como migro sem quebrar'

Sinalizar depreciação no contrato

HTTP

```
HTTP/1.1 200 OK
Content-Type: application/json
Deprecation: true
Sunset: Sat, 30 Nov 2026 23:59:59 GMT
Link: <https://api.lunalog.com/v2/rotas>; rel=successor-version

{ 'rotas': [ ... ] }
```

Vinte e três integrações, oito presas na v1

- **LunaLog tem 23 transportadoras integradas via API**
 - Oito ainda dependem da v1 lançada em 2021

LUNALOG

A LunaLog tem 23 transportadoras integradas pela API, e oito delas ainda usam a v1 de 2021. Manter duas versões em paralelo tem custo crescente: cada bug corrigido precisa ser portado para as duas, e cada mudança de negócio exige implementação duplicada. A equipe gastava mais energia sustentando o passado do que construindo o futuro. A virada veio com uma política de sunset: seis meses de aviso formal, suporte ativo à migração e uma data de desligamento definida. O versionamento deixou de ser dívida crescente e virou processo controlado.

Estratégias de compatibilidade

- **Janela de depreciação: anunciar o fim com prazo claro**
 - Aviso formal, data de sunset e suporte à migração
- **Duas versões vivas sem duplicar lógica de negócio**
 - Camada fina de tradução sobre um núcleo único
- **Consumer-driven contracts detectam quebra antecipadamente**
 - Os testes do consumidor rodam contra cada mudança sua
- **Versionamento explícito por URI, header ou media type**

Duas versões para sempre é a armadilha

- Cada versão viva multiplica o custo de manutenção
- Sem data de sunset, a v1 nunca morre

Manter versões antigas indefinidamente parece gentileza com o cliente, mas é dívida que cresce sozinha. Toda versão precisa nascer com um plano de aposentadoria, ou ela vira eterna.



Você sabe evoluir contratos sem quebrar consumidores. Na penúltima aula, vamos juntar todas as estratégias de migração do curso, Strangler Fig, expand-contract, feature flags, dual-write, em um único plano coerente. Porque modernização sem plano é só refatoração com otimismo.

Migrações incrementais em produção